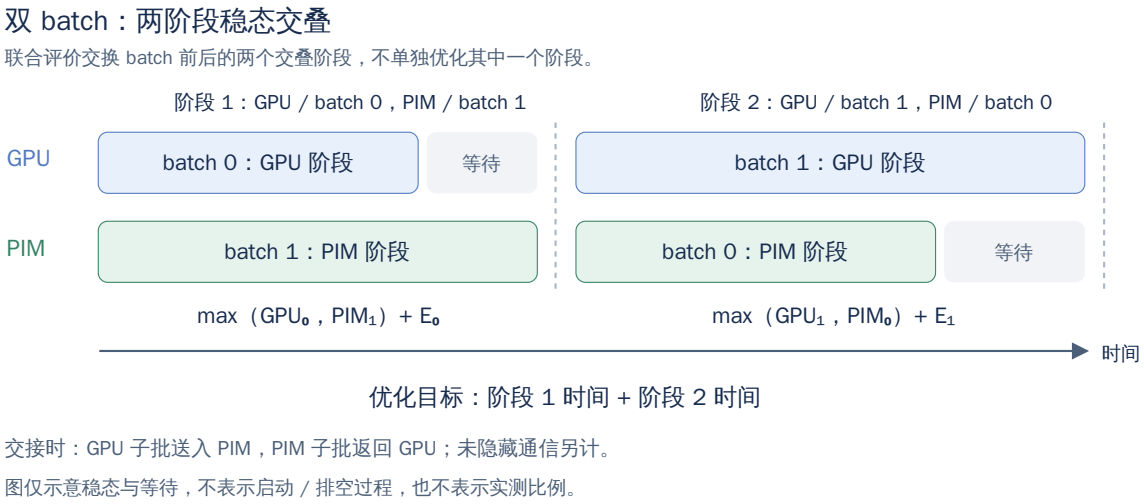
异构 AF 推理的成本模型与动态映射

> 本文给出两阶段稳态目标、算子成本与动态调整原则；硬件成本尚待校准，完整策略尚未接入模拟器。

1. 优化目标：连续两个交叠阶段的总时间

系统由 GPU 侧 G 和 DIMM-PIM/PNM 侧 A 组成，两个 decode 子批 $\mathcal{B}_0, \mathcal{B}_1$ 交替执行。固定 AF 映射将 Attention 放在 A，将投影和 FFN 放在 G；动态映射 m 改变各请求的算子边界及 Attention 分工，但不改变两阶段流水结构。



两个交替阶段的联合目标

图 1：先由 batch 0 使用 GPU、batch 1 使用 PIM，再交换角色。优化对象是这两个交叠阶段的总时间，而不是其中某一个最大值。

令 $T_{d,b}(m)$ 表示子批 b 在设备侧 d 的执行时间，则一个稳态 AF 周期的成本为：

$$\begin{aligned} T_{\mathrm{AF}}(m) = {} & \max(T_{G,0}(m), T_{A,1}(m)) \\ & + \max(T_{G,1}(m), T_{A,0}(m)) + E_0(m) + E_1(m). \end{aligned} \tag{1}$$

前两项分别是两个交叠阶段的瓶颈时间；E_0, E_1 是各阶段未隐藏的边界通信与争用影响。某次调整可以缩短第一个阶段而延长第二个阶段，只有总和减少才改善这个周期。两个 batch 不能合并成一次理想的大 batch 后再估价。

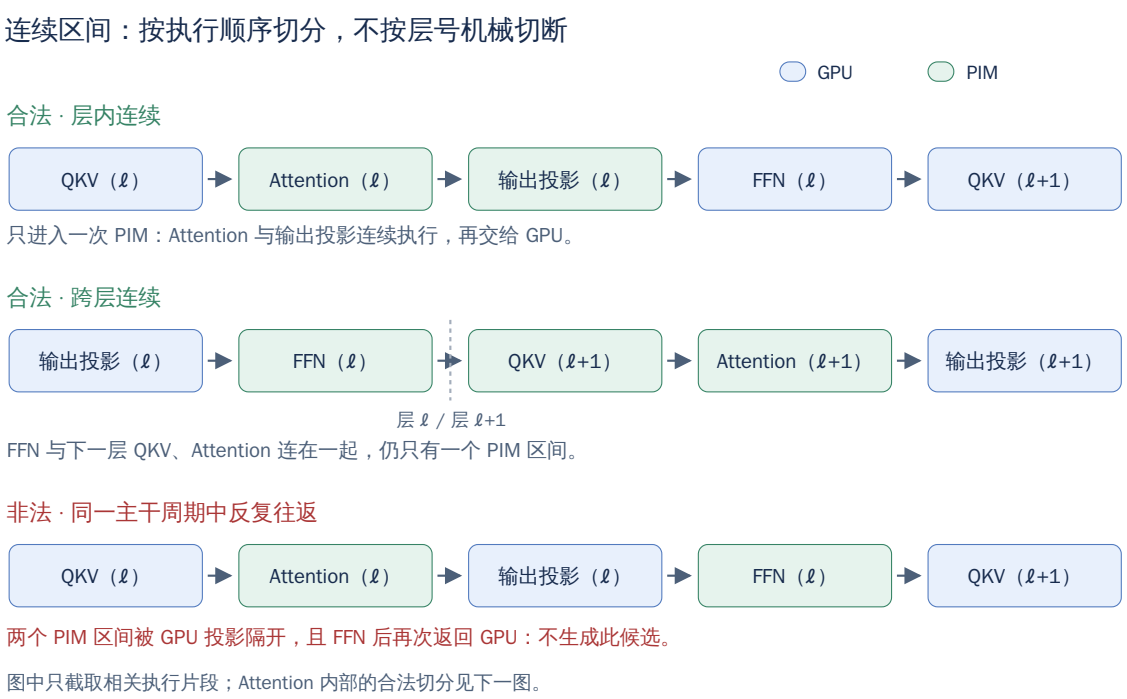
一轮 decode 指两个子批中的活跃请求各生成一个 token。把该轮经过的 AF 周期依执行顺序编号为 k，得到当前负载下的基础目标：

$$T_{\mathrm{step}}(m) = \sum_{k} T_{\mathrm{AF},k}(m), \qquad \min_{m \in \mathcal{F}} T_{\mathrm{step}}(m). \tag{2}$$

$\mathcal{F}$ 是满足连续区间、数据和设备约束的映射集合。k 对应重复的层间执行周期，跨层游标按真实层号归组；首尾层、输出头和采样计入相应阶段。同构内部周期可复用成本。式（2）不展开流水线启动和排空；后续动态决策在此基础上加入请求变化和迁移成本。双子批交叠与设备延迟预测参考 CHIME。[1]

2. 映射表示：连续区间与显式 Merge 算子

对每个请求，PIM 只执行一个连续区间；每周期前台数据仅进行一次 G→A 和一次 A→G 交接，不增加往返。区间可以跨层，例如本层 FFN 与下一层 QKV、Attention 连续放在 PIM。不同请求可以采用不同区间。



连续区间的合法性

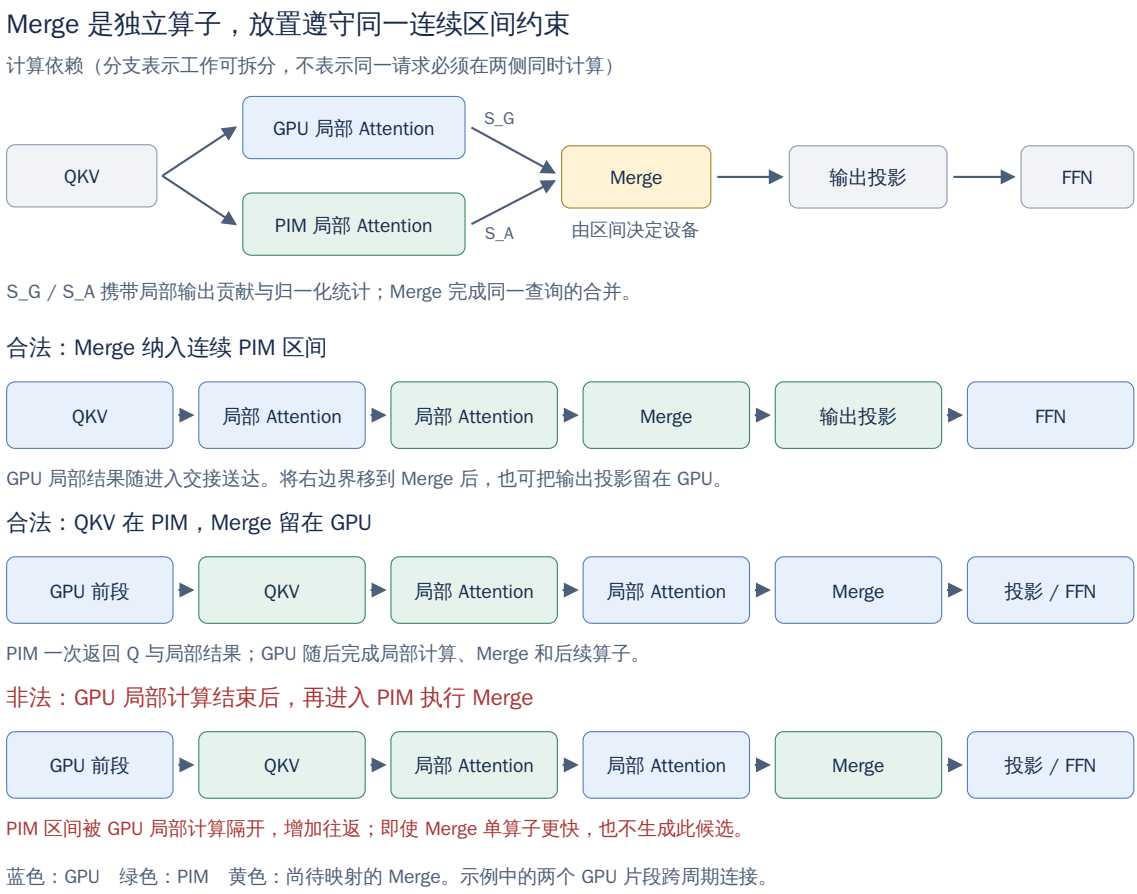
图 2：连续性沿真实执行顺序判断。被 GPU 算子隔开的两个 PIM 区间不合法。首版保留两侧共同参与，FFN 按请求完整分配，不搜索整请求单侧执行或逐层独立边界。

Attention 按 KV-token 位置切分。令 $\mathcal{K}_r$ 为请求 r 本次使用的 token 位置集合，切分满足：

$$\mathcal{K}_{r,G} \dot{\cup} \mathcal{K}_{r,A} = \mathcal{K}_r, \qquad \mathcal{K}_{r,d}^{(h)} = \mathcal{K}_{r,d} \quad \text{对所有 head } h. \tag{3}$$

第一项表示两侧的 KV 贡献互不重叠且覆盖全部选中 token；第二项表示同一请求的所有 head 使用同样的切分。不同请求的集合可以不同。稠密模式使用全部历史，稀疏模式由固定选择器提供 $\mathcal{K}_r$；头数与设备内头映射是配置，不是搜索维度。

两侧局部 Attention 产生 S_G, S_A，然后执行独立的算子 $\mathrm{Merge}(S_G, S_A)$，输出交给输出投影。Merge 与 QKV、局部 Attention、输出投影和 FFN 一样参与映射，必须遵守输入依赖和连续 PIM 区间约束。



将 Merge 作为独立算子

图 3：上方是计算依赖，下方是合法与非法放置示例。Merge 放入 PIM 区间时，GPU 局部结果必须随已有的进入交接送达；若 GPU 局部计算只能在 PIM 返回 Q 后执行，Merge 就必须留在 GPU。无需另立一套合并位置规则。

S_G, S_A 包含局部输出贡献及 softmax 归一化统计，Merge 进行精确的归一化合并，不是直接相加两个输出向量。[2] 完整 Attention 在一侧完成时，不产生额外的跨侧 Merge 调用。归一化、残差和必要新 KV 随真实依赖处理；缺失数据先准备，不能用一次临时补交破坏两阶段结构。

3. 从算子与数据布局得到设备执行时间

3.1 阶段时间就是实际算子调用的累加

给定映射 m，按执行顺序遍历设备侧 d 为子批 b 执行的算子。同一层、同一算子、同一就绪阶段的请求聚合成一次调用：

$$T_{d,b}(m) = \sum_{o \in \mathrm{Calls}(m,d,b)} f_{d,o}(b_o, X_o). \tag{4}$$

b_o 是实际参与该调用的请求数，X_o 包含矩阵维度及数据布局，$f_{d,o}$ 返回调用延迟。式（4）相当于一个循环：收集参与请求，确定输入形状，查询延迟，再累加。它不是把每个请求单独运行的延迟相加；调用内部已有的批处理和硬件并行由 f 表达，不额外搜索同一设备上的任意算子并发。

例如，8 个请求中有 3 个在 PIM 执行输出投影、5 个在 GPU 执行，就分别计算 batch 3 和 batch 5 的投影时间。若之后 8 个请求的 FFN 都在 GPU 且已经就绪，FFN 可以按 batch 8 计算。层号或就绪阶段不同则分开。

3.2 投影与 FFN：每个请求贡献一行输入

一次 decode 中，参与线性算子的每个请求贡献一个 token。将这些输入堆叠后，矩阵乘法及其主算量为：

$$X_{M\times K}W_{K\times N}=Y_{M\times N},\qquad C_{\text{linear}}=2MKN. \tag{5}$$

M 是本次参与请求数，K,N 是该算子的输入、输出维度，W 是已驻留权重。输出投影和 QKV 投影都按这个形式计费；FFN 则由 up、gate、down 等矩阵调用及必要逐元素操作组成。按请求分配整个 FFN 不改变其内部维度，只改变这些调用的 M。

权重布局及设备组内并行方式预先固定。将算子交给另一侧时，若那里缺少权重，先支付一次复制与布局准备成本；后续调用仍需按该侧的实际内核计入权重读取时间。复制权重和执行 GEMM 是两种不同成本。

3.3 Attention：私有 KV 与共享 KV 决定矩阵的行数

先看一个 head。对于某段长度为 L_p 的相同 KV，将本次访问它的 M_p 个请求的 Q 堆叠起来：

$$\begin{aligned}
Z_p &= Q_pK_p^{\top}, && (M_p\times d_h)(d_h\times L_p)\longrightarrow M_p\times L_p,\\
P_p &= \mathrm{softmax}_{\text{row}}(Z_p/\sqrt{d_h}),\\
O_p &= P_pV_p, && (M_p\times L_p)(L_p\times d_h)\longrightarrow M_p\times d_h.
\end{aligned} \tag{6}$$

d_h 为 head 维度。私有 KV 的 $M_p=1$，QK 和 PV 是矩阵向量运算；共享前缀的 M_p 是本次调用中引用它的请求数，可以形成 GEMM。不同请求的私有 KV 不相同，即使长度相等，也只能做一组独立的 Attention，不能把它们当作共享同一个 K/V 的 GEMM。

softmax 仍逐请求、逐行计算，不能在请求之间归一化。式（6）只算这一段 KV 的局部结果，同时保留归一化统计；不同 KV 段的结果经精确合并得到该请求的 Attention。设备内多段的合并计入局部调用，跨设备结果由第 2 节的 Merge 算子处理。共享前缀上的查询聚合与这种分解可参考 Hydragen。[3]

对于这一段 KV，所有查询 head 上的 QK、PV 主算量为 $4h_qM_pL_pd_h$，h_q 为查询头数。KV 容量按实际 KV 头数、长度和精度计算；实际读取字节由内核的分块与复用决定。共享一份 KV 不等于所有查询只做一次计算，也不保证内存只读一遍。

据此，局部 Attention 的估价流程是：

```
收集该设备本次需要计算的 KV 段
对引用相同 KV 段、且在同一子批和就绪阶段的查询进行聚合
    共享段：M = 参与引用者数
    私有段：每个请求各自 M = 1
根据各段长度、查询数和物理位置，估计实际 Attention 调用
计入 softmax、设备内合并等成本，返回本次调用延迟
```

这里描述计算内容，不要求逐段单独启动内核。GPU 可以批量执行多个私有 Attention；PIM 可以在不同执行组并行处理。若采用融合内核，直接使用覆盖这些步骤的成本，不再重复累加其内部 QK、softmax 和 PV。

3.4 硬件成本：矩阵形状相同，能使用的资源未必相同

成本接口暴露以下参数。其中硬件配置与内核在一次决策中固定，KV 的物理分布随候选更新：

参数	用途
设备数量、组内并行方式及独立执行组	确定算子如何分片、哪些工作能够并行
各执行组有效算力 P^{eff}、带宽 B^{eff}	估计特定矩阵形状、精度和布局下的计算与访存时间
GPU/PIM 可用容量	检查权重、KV、副本及临时空间能否放下
跨设备和必要组内链路的带宽、启动开销	估计传输与归约
权重布局、KV 的物理分布和支持的内核	确定实际访存量、可用资源和必要转换

GPU 使用固定 GPU 数量、TP 等并行配置下的 profile 曲线或查表：输入包含 GEMM 形状，或 Attention 的查询组、KV 长度及布局；输出包含该配置必要的组内通信。缺少测量时，可先以 roofline 估计。PIM 的矩阵内核也从同一基本形式出发：

$$t_g(u) = \alpha_{g,u} + \max\left(\frac{C_{g,u}}{P^{\text{eff}}_{g,u}}, \frac{D_{g,u}}{B^{\text{eff}}_{g,u}}\right). \tag{7}$$

u 是在执行组 g 上执行的矩阵内核，C, D 是分配给该组的算量和实际访存字节，α 是启动等固定开销。有效算力、带宽随形状和布局离线校准；softmax、归约等不适合简单 FLOPs 估计的步骤用小型成本表。不能因二者都做 GEMM，就给 GPU 与 PIM 相同的效率。

关键差别在于数据分布。权重按照固定方案分片或复制；Attention 的 KV 则由运行时块表决定所在 DIMM/rank。将一个算子调用的工作按实际可并行执行组归组，可写为：

$$f_{A,o} = \max_{g \in \text{Active}(o)} \left(\sum_{u \in \text{Work}(o,g)} t_g(u) \right) + t_{\text{sync},o}. \tag{8}$$

$\text{Work}(o, g)$ 是该调用分到组 g 的工作，组内按固定顺序累加；能够独立执行的组之间取最大值。这里矩阵工作的 t_g 使用式（7），softmax 等辅助工作则查表。$t_{\text{sync},o}$ 是未包含在组内成本中的必要跨组同步或合并。若组间共享总线、不能同时执行，就必须按固定硬件规则串行计入或修正有效带宽，不能直接套理想并行的最大值。

例如，一段 KV 只在 DIMM 0，处理它不能使用 DIMM 1 的空闲带宽；多个请求集中在同一 DIMM 也会形成局部瓶颈。只有数据按可执行布局分布到多个组，且计算确实并行时，才获得相应带宽。CHIME 的运行时模型按各 rank 的工作量预测、由最慢 rank 决定 PIM 时间，为这种建模提供了直接参考。[4]

因此，首版可以对固定布局下的私有 Attention 使用按 token 数拟合的简式；共享前缀和 FFN 则保留 M 变化对矩阵效率的影响。需要校准的是“形状 + 布局 + 参与资源”到延迟的映射，而不是构造在线硬件事件模拟器。

3.5 边界通信

沿每个请求的连续区间提取跨边界仍需要的张量，再按子批打包：

```
G→A：PIM 区间需要、但尚未在 PIM 就绪的前台输入
A→G：GPU 接续需要、由本次 PIM 区间产生的结果
```

张量可能是隐藏状态、Q、新 KV、局部结果、Merge 输出或残差。移动算子边界会改变这些包的内容。对于一份非空传输包：

$$t_{\text{link}}(D) = \alpha_{\text{link}} + D/B_{\text{link}}. \tag{9}$$

D 为打包字节，α_{link} 为提交开销，B_{link} 为实际路径带宽。将固定重叠规则下未隐藏的部分计入式（1）的 E_0, E_1；DIMM 通信与计算争用同样计入，但不和设备曲线重复收费。未经校准时不默认完全隐藏。历史 KV 与权重的一次性准备另在第 5 节计费。

4. 运行时状态：谁引用哪段 KV，数据实际放在哪里

运行时需要关联请求、逻辑 KV 和物理副本，而不仅是记录“GPU 承担多少比例”。

记录	必需字段
请求表	所属子批、当前长度、预测剩余生成长度、连续算子区间、引用的 KV 块、后续新 KV 的写入侧
KV 块表	逻辑块标识及 token 范围、各层对应的物理地址/分片、字节数、GPU/DIMM/rank 位置、布局与就绪状态，以及当前计算归属
引用关系	每个块被哪些活跃请求引用、引用数；由此找到本次实际参与计算的查询
权重与准备状态	两侧已有权重、在途副本、目标空间预留及安全切换状态

前缀树可以维护 token 前缀到共享逻辑块的映射，请求沿树路径引用这些块；物理块表再回答这些数据位于哪台设备、哪个 DIMM/rank。两者分工不同：前缀树解决“共享谁”，物理块表解决“放在哪”。SGLang 的 RadixAttention 提供前缀树与活跃引用计数的参考；异构副本、计算归属及引用者查询索引由本系统补充。[5]

例如，某层共享前缀 p 被请求 r_1, r_2, r_3 引用，其中 r_1, r_2 在 batch 0，r_3 在 batch 1。即使引用数为 3，两次调用的 M_p 分别是 2 和 1，不能跨两个交叠阶段合成 $M_p = 3$。实际聚合还要满足该调用的就绪条件：

```
p 的本次查询集合 =
    引用 p 的请求 ∩ 当前子批 ∩ 本次已就绪且需要访问 p 的请求
M_p = 这个集合的大小
```

若将 p 从 A 复制到 G，缺少目标副本时只复制一份；安全切换后，引用者对这段 KV 的计算统一改到 G。需要联合检查这些请求的连续区间与 Merge 放置。副本可以暂时同时存在，但一次查询中的每段 KV 只指定一个计算侧，不重复贡献结果。

请求到达、结束、生成新 token 或块复制完成时，增量更新上述表。预测未来某轮负载时，只保留预计尚未结束的引用者，重新得到 M_p，并更新私有后缀长度。已经结束但仍保留在 prefix cache 中的块占容量，却不因缓存存在而产生 decode 计算。

共享关系以实际相同 KV 为准：多个请求共用已有前缀，不代表它们各自新生成的后缀也可以共享。新的共享关系在实际前缀命中或分支复用时建立，不预设未来不同请求会生成相同内容。

5. 动态调整：比较未来执行时间与准备成本

5.1 首先改变未来 KV 的写入位置

以当前 KV 全在 A 的请求为例，在当前长度处设置分界，让后续新 KV 写入 G，并由 G 处理这部分 Attention。旧历史不动，每轮两侧局部结果按既定数据流合并。



只改变未来 KV 写入位置时的负载演化

图 4：在请求持续活跃、稠密 Attention 且暂不写回的示例中，经过 j 轮后，旧历史仍由 A 扫描，新增的 j 个 KV token 由 G 扫描。相对于继续全部写入 A，转移的是逐渐增长的后续工作，不是立刻减少 A 的既有历史负载。

这种动作没有历史 KV 的准备流量；新 KV 尚未产生，不需要先存到 A 再迁到 G。它仍可能产生新 KV 从生成侧到存储侧的必要传输，以及 Q、局部结果与 Merge 开销，这些逐轮进入第 3 节成本。GPU 容量也随新增 KV 增长，不能把“无历史复制”当作无成本。

反向调整同理。若采用近期 KV 暂留、成批写回的规则，预测过程还要按该固定规则更新两侧块表，并计入实际写回；它是可选实现方式，不是所有请求必须采用的动作。

5.2 历史 KV 与线性算子提供更快的工作量调整

只改变增长方向可能起效太慢：请求即将结束，或当前历史负载远大于近期新增负载。这时可以使用目标侧已有副本，或付费复制一段历史 KV，使对应计算在数据就绪后立即改由目标侧执行。

高复用前缀是值得优先检查的对象。一份共享 KV 的复制可以改变多个请求的计算位置，而不是逐请求复制相同数据。但最终应比较两侧实际调用时间：源侧可能已经通过查询聚合充分复用了读取，目标侧也可能因 GEMM 效率或局部资源不足而接手得不划算。引用数说明能够影响多少工作，不直接等于时间收益。

当 GPU 的投影或 FFN 成为瓶颈、A 侧有余量时，则对部分请求扩展连续 PIM 区间，接手相邻投影或完整 FFN。先准备缺失权重，GPU 原件可以保留；之后调整请求分工无需反复复制已有权重。相反方向可以收回区间。所有动作都遵守同一连续性和数据就绪约束。

5.3 用一个短窗口比较这些动作

每隔 Δ 个完整 decode 轮重新决策，评价未来 H 轮；例如每 16 轮触发一次，但触发间隔不必等于预测窗口。设请求 r 的预测剩余长度为 $\widehat{R}_r$，未来第 j 轮仅计入 $\widehat{R}_r \geq j$ 的请求。每个候选按自己的新 KV 写入规则更新两侧长度和共享查询组，再用式（4）至（9）求设备与通信成本，用式（1）、（2）得到该轮总时间。

将候选动作或动作组合记为 a，评价：

$$V(a) = \sum_{j=1}^{H} [T_{\text{step}}(\text{keep}, j) - T_{\text{step}}(a, j)] - C_{\text{exposed}}(a). \tag{10}$$

keep 表示延续当前映射和写入规则，同样随请求进度演化；$T_{\text{step}}(a, j)$ 表示采用候选后的第 j 轮时间。求和项是整个窗口内的预计时间节省，C_{exposed} 是一次性准备与控制造成的未隐藏时间。采用剩余长度的点预测时，这是条件于该预测的估计，不另建复杂的概率模型。

这里的候选是一条随 decode 推进的数据放置与执行轨迹，不是把某个未来时刻的平衡程度复制到整个窗口。只改新 KV 去向时，收益随新 KV 积累逐轮计算；历史复制或权重准备未完成时，仍按旧映射计时，就绪后才计新映射收益。窗口内不假设后续还有额外优化动作，下个决策点再用新状态重新估价。

准备按目标缺失的物理数据去重：

```
准备字节 = 缺失历史 KV 的字节 + 缺失权重的字节
准备成本 = 这些数据的复制、必要布局转换和安全发布造成的暴露时间
```

共享前缀被十个请求引用也只准备一份；准备同一权重供多个请求使用也只计一次。已存在的可执行副本不再复制。后台准备若能重叠，会推迟新映射生效，但不全额扣除复制历时；若它阻塞或拖慢前台，则将额外延长计入 C_{exposed}。例如，完全阻塞式准备发生在下一轮前，就扣这段停顿；若后台准备覆盖两轮旧执行，则前两轮不计新收益，只另扣未隐藏干扰。准备成本与逐轮通信不重复计费。

前缀复用自然包含在式（10）中：目标缺失数据只准备一份，而每轮被影响的查询数由引用请求和预测存活状态共同决定。例如两个引用者预计还能生成 1 和 16 个 token，立即生效时窗口内共有 17 次使用，不是 32 次；复制就绪晚了，还需扣除生效前的使用机会。无需额外叠加一项“前缀收益”。

6. 搜索原则与一个贪心框架

搜索目的不是枚举全部映射，而是围绕两个交替阶段的瓶颈提出少量有意义的动作。首先检查无历史搬运的调整，再检查需要准备的调整；这是候选生成的顺序，不意味着有一个正收益的无复制动作就拒绝比较其他动作。

候选	改变什么工作	主要取舍
改变后续新 KV 的写入侧	将新增 KV 对应的后续 Attention 逐渐交给另一侧	无历史复制，但起效渐进，受剩余长度与容量影响
使用已有历史 KV 副本	改变对应 KV 的计算侧	无新增历史复制，仍需满足边界、布局与副本就绪要求
复制部分历史 KV	接手已经存在的 Attention 工作	起效较直接，但需支付准备成本；优先检查高复用块及瓶颈 DIMM 上的块
扩展或收回连续算子区间	在请求粒度转移相邻投影或完整 FFN	考虑权重副本、改变后的 batch 效率、Merge 和边界通信

共享前缀按对象及其引用者联合调整；私有历史按已有 KV 分界附近的块调整；请求组不枚举任意子集。同一请求所有 head 同步改变切分。暂不固定块步长、请求组大小或排序分数，这些需要后续结合成本曲线和搜索开销确定。

```
best ← 不调整
重复，直到没有改进或达到决策开销上限：
    分别查看 best 在两个交替阶段的当前及预测瓶颈
    在 best 基础上，围绕这些瓶颈生成少量调整候选：
        改变新 KV 去向；利用已有副本
        复制选中历史 KV；调整连续算子边界
    用复用关系、剩余寿命、准备字节和物理位置筛选候选
    对每个候选：
        检查连续区间、依赖、硬件支持、容量与服务约束
        从实际当前状态出发，确定去重后的准备与生效过程
        向前估计 H 轮：请求存活、新 KV 落点、共享查询组
        计算完整两阶段时间节省，再扣一次暴露准备成本
    若某个候选的完整净收益高于 best，则更新 best
    否则停止
若 best 的净收益超过误差裕量，则准备并在安全边界发布
否则保持当前策略
```

每次比较都相对实际当前状态，不能将多次局部调整的收益相加。候选组合可能共享权重或 KV 准备，也可能把接手侧再次变成瓶颈，因此必须重算组合后的两阶段时间。无变化调用可以缓存，只增量更新受影响的请求组和执行组。

筛选时需要遵守以下原则：

- 连续性和数据依赖是硬约束，不能靠新增 AF 往返完成候选。
- 容量按实际 GPU/DIMM 分片检查，包含权重原件与副本、KV 副本、在途预留、临时空间及窗口内增长；预测退出不授权提前释放真实数据。
- 在收益相近时偏向较少准备字节、较少状态变化的动作，而不是只优化搬运量。
- 高复用、较长剩余寿命、已有副本、位于瓶颈执行组都是筛选线索；最终收益仍由式（10）决定。

- 不因一个阶段变快、瞬时利用率更接近或某个单算子加速，就直接接受调整。

目前确定的是动作空间与统一评价口径。候选排序、步长、窗口长度、搜索停止阈值和误差裕量仍待细化；上述贪心框架不保证全局最优，也不把这些策略参数提前视为已验证结论。

7. 动态输入与下一步

新请求在 prefill KV 就绪后按固定规则加入子批，默认使用 AF 映射；命中已有共享前缀时继承其计算归属并检查区间合法性。接纳和真实结束更新引用关系，下一周期据此调整；显著变化可以提前触发，但限制触发频率。未知未来到达不纳入首版预测。

稀疏模式将固定选择器得到的访问块作为成本输入：每个查询只对实际选中的块贡献计算，块的存储引用数不直接等于当轮访问数。所有 head 的切分约定不变，未来选集用当前可见信息或固定近期规则估计，不引入未来真实掩码。

Serving 以预计完整轮时间、已等待时间和迁移停顿检查服务约束；该稳态模型不是尾延迟保证，接纳和超载处理仍由外层机制负责。在线、离线沿用同一成本模型，再分别检验服务指标和任务完成时间。

接下来优先落实三项：实际算子调用及 KV 引用统计；带物理布局的 GPU/PIM 成本接口；新 KV 定向与历史复制的短窗口比较。必要小例子先验证计算与计费正确性，再细化搜索，不急于报告端到端收益。已有原语验证不代表本文完整决策已实现；硬件布局与支持继续维护在技术综述。

参考文献

---

1. CHIME，§II-B 的子批交叠、§VI-A 的 rankset 通信重叠、§VI-C 的 GPU/PIM 运行时预测。其 PIM 时间按各 rank 预测并取最长者；本文共享查询及 FFN 的成本仍需另行校准。本地 PDF；原文。↩
2. FlashAttention 的分块归一化统计提供精确合并依据。本地 PDF；原文。↩
3. Hydragen，§3.1–3.3 的前缀分解、跨请求查询聚合与分层共享。原文。↩
4. CHIME，§II-B 的子批交叠、§VI-A 的 rankset 通信重叠、§VI-C 的 GPU/PIM 运行时预测。其 PIM 时间按各 rank 预测并取最长者；本文共享查询及 FFN 的成本仍需另行校准。本地 PDF；原文。↩
5. SGLang，§3 RadixAttention 的前缀树、KV 映射与活跃请求引用计数。本文的异构驻留与计算归属是额外设计，不归于原论文。原文。↩